

CEFET/RJ

Centro Federal de Educação Tecnológica Celso Suckow da Fonseca

Bacharelado em Ciência da Computação

GCC1734 – Inteligência Artificial

Aprendizado de Máquina

Notas de Aula

Prof. Eduardo Bezerra

Escola de Informática e Computação

CEFET/RJ

16 de maio de 2026

Sumário

1	Motivação e Contexto	4
1.1	O que é aprendizado de máquina?	4
1.2	Hipótese de aprendizagem	5
1.3	Relação com outros paradigmas de IA	5
1.4	Taxonomia do aprendizado de máquina	5
1.5	Quando usar aprendizado de máquina?	6
2	O Framework de Aprendizado Supervisionado	8
2.1	Dados, tarefas e modelos	8
2.2	O espaço de hipóteses	8
2.3	Função de perda	9
2.4	O problema fundamental: generalização	10
2.5	Divisão dos dados: treino, validação e teste	11
3	Modelos Lineares	12
3.1	Regressão linear	12
3.1.1	Exemplo resolvido	12
3.2	Gradiente descendente	13
3.2.1	Exemplo resolvido: um passo de gradiente descendente	14
3.3	Regressão logística e classificação binária	15
3.4	Engenharia de características	16
4	Generalização: Viés, Variância e Regularização	17
4.1	Underfitting e overfitting	17
4.2	O dilema viés-variância	17
4.3	Regularização	18
4.3.1	Exemplo resolvido	20
4.4	Validação cruzada	20
4.5	Métricas de avaliação	20
5	Árvores de Decisão e Métodos Ensemble	22
5.1	Árvores de decisão	22
5.2	Entropia e ganho de informação	23
5.2.1	Exemplo resolvido	23
5.3	Controle de complexidade e poda	24
5.4	Métodos ensemble	24
6	Redes Neurais Artificiais	26
6.1	Do perceptron à rede multicamadas	26
6.2	Funções de ativação	27
6.3	Retropropagação	28
6.4	Aprendizado profundo: uma visão geral	29
7	Aprendizado Não Supervisionado	30
7.1	Quando não temos rótulos	30
7.2	k -means	30
7.2.1	Exemplo resolvido	31

7.3	Análise de Componentes Principais (PCA)	32
8	Resumo	33
9	Atividades Intra-Classe	34
9.1	Aquecimento	35
9.2	Regressão linear e gradiente descendente	35
9.3	Entropia e construção de árvores	36
9.4	Diagnóstico de generalização	36
9.5	k -means: execução manual	36
9.6	Perguntas para discussão em sala	37

Objetivos

Ao final da leitura, você deve ser capaz de:

Motivação e taxonomia

- Explicar o que é aprendizado de máquina e posicioná-lo em relação a busca, planejamento e aprendizado por reforço.
- Distinguir os três paradigmas principais (supervisionado, não supervisionado e por reforço) e identificar o paradigma adequado a um problema dado.
- Enunciar a hipótese de aprendizagem e reconhecer seus limites.

Framework de aprendizado supervisionado ★

- Formalizar um problema de aprendizado supervisionado como a busca de uma hipótese $h \in \mathcal{H}$ que minimiza uma função de perda $\mathcal{L}$.
- Distinguir erro de treinamento de erro de generalização e explicar por que minimizar o primeiro não garante o segundo.
- Justificar a necessidade de divisão dos dados em conjuntos de treino, validação e teste.

Modelos lineares e gradiente descendente ★

- Enunciar o modelo de regressão linear e a função de perda quadrática e executar um passo de gradiente descendente manualmente.
- Descrever o modelo de regressão logística e interpretar sua saída como probabilidade.
- Explicar o papel da engenharia de características na extensão da capacidade de modelos lineares.

Generalização, viés-variância e regularização ★

- Identificar sintomas de underfitting e overfitting em curvas de validação.
- Enunciar o dilema viés-variância e relacioná-lo à complexidade do modelo.
- Descrever como a regularização ℓ_1 e ℓ_2 penaliza modelos complexos e previne overfitting.
- Descrever a validação cruzada k -fold e explicar seu propósito.

Árvores de decisão e métodos ensemble ★

- Descrever o algoritmo de construção de árvores de decisão via ganho de informação.
- Calcular a entropia de uma distribuição e o ganho de informação de um atributo de divisão.
- Explicar como Random Forests e boosting reduzem viés e variância.

Redes neurais artificiais ★

- Descrever a arquitetura de um perceptron multicamadas e o papel das funções de ativação.
- Explicar intuitivamente como a retropropagação calcula gradientes via regra da cadeia.
- Conectar o treinamento de redes neurais ao gradiente descendente estocástico.

Aprendizado não supervisionado

- Descrever o algoritmo k -means e executá-lo sobre um exemplo simples.
- Explicar intuitivamente o objetivo da análise de componentes principais (PCA).

Roteiro de leitura

Estas notas apresentam uma visão panorâmica do aprendizado de máquina, com foco nos conceitos e algoritmos mais centrais para a continuidade da disciplina. O material está organizado em torno de cinco aulas:

- **Aula 1** (Seções 1–2): motivação, taxonomia e o framework de aprendizado supervisionado.
- **Aula 2** (Seção 3): modelos lineares e gradiente descendente.
- **Aula 3** (Seção 4): generalização, viés-variância, regularização e avaliação.
- **Aula 4** (Seção 5): árvores de decisão e métodos ensemble.
- **Aula 5** (Seções 6–7): redes neurais e aprendizado não supervisionado.

Os objetivos marcados com ★ são os mais centrais: concentre-se neles se o tempo for limitado. O material pressupõe familiaridade com busca, planejamento e aprendizado por reforço, abordados nas notas anteriores.

1 Motivação e Contexto

1.1 O que é aprendizado de máquina?

Nos módulos anteriores, estudamos agentes que agem sob diferentes graus de conhecimento sobre o ambiente. Algoritmos de busca como A* recebem um modelo completo e encontram soluções ótimas. Minimax e MCTS raciocinam sobre um modelo adversarial fornecido. O aprendizado por reforço vai além: o agente não recebe o modelo, mas aprende uma política interagindo com o ambiente e recebendo recompensas.

O *aprendizado de máquina* (AM, ou *machine learning*) aborda um problema diferente, embora relacionado: aprender a partir de *dados*. Em vez de um modelo do ambiente ou de um ciclo de ação-recompensa, o agente recebe um conjunto de exemplos de entrada-saída (ou apenas de entradas) e deve induzir, a partir deles, uma função que generalize para novas situações.

A pergunta central do AM pode ser formulada assim: *dado um conjunto de dados sobre um fenômeno, como um sistema pode extrair automaticamente regularidades que lhe permitam tomar boas decisões em situações ainda não vistas?*

Essa pergunta é relevante em uma gama surpreendentemente ampla de problemas: classificar e-mails como spam ou não spam; prever o preço de um imóvel; reconhecer a fala humana; detectar fraudes bancárias; recomendar filmes; diagnosticar doenças a partir de imagens. Em todos esses casos, seria muito difícil (frequentemente impossível) escrever manualmente um conjunto de regras que capture o fenômeno. O AM oferece uma alternativa: deixar que o algoritmo infira as regras a partir dos dados.

1.2 Hipótese de aprendizagem

Aprender a partir de dados só faz sentido se os dados observados carregam informação sobre casos futuros. Essa é uma suposição forte. Ela não vem do algoritmo em si; vem do problema e do modo como os dados foram coletados. O AM se apoia nessa suposição de forma explícita, na forma da *hipótese de aprendizagem*.

Hipótese de aprendizagem

Os dados de treinamento e os dados futuros devem ser amostras do mesmo processo gerador, ou pelo menos de processos suficientemente semelhantes. Em termos práticos, espera-se que as regularidades observadas nos dados de treinamento também estejam presentes nos dados em que o modelo será usado.

Em outras palavras: *o passado é informativamente semelhante ao futuro*.

Essa hipótese é o alicerce estatístico de todo o AM. Se ela for satisfeita, um modelo que aprende bem dos dados de treinamento provavelmente se sairá bem em dados novos. Se ela for violada — por exemplo, se os dados de treinamento forem uma amostra tendenciosa do fenômeno, ou se a distribuição mudar ao longo do tempo — o modelo aprendido pode generalizar mal, mesmo que seu desempenho no conjunto de treinamento seja excelente.

A hipótese de aprendizagem não é verificável em abstrato: ela depende de como os dados foram coletados e do problema em questão. Parte do trabalho de um praticante de AM é avaliar criticamente se essa hipótese é razoável para um dado conjunto de dados.

1.3 Relação com outros paradigmas de IA

O AM não substitui os outros paradigmas de IA; ele se combina com eles de formas precisas. A Tabela 1 resume as diferenças mais relevantes.

Há também conexões profundas entre os paradigmas. O aprendizado por reforço com aproximação de funções (abordado nas notas anteriores) é uma instância de AM aplicada ao problema de estimar $Q^*(s, a)$: as “características” $f_i(s, a)$ são exatamente o tipo de representação que o AM usa, e a regra de atualização dos pesos $w_i \leftarrow w_i + \alpha \delta f_i(s, a)$ é uma instância de gradiente descendente sobre o erro temporal, exatamente o algoritmo que estudaremos na Seção 3.

Por que AM agora?

As notas de aprendizado por reforço introduziram o conceito de *aproximação de funções*: em vez de uma tabela, o agente aprende um vetor de pesos que comprime o conhecimento. Aquilo era AM aplicado a AR. Nestas notas, estudamos AM de forma mais ampla: os mesmos mecanismos (representação, otimização, generalização) surgem em problemas onde o sinal de aprendizado não são recompensas, mas exemplos rotulados ou dados não rotulados.

1.4 Taxonomia do aprendizado de máquina

A Figura 1 mostra a organização dos principais paradigmas e algoritmos de AM. Esta leitura cobre os itens destacados.

Tabela 1: Comparação entre paradigmas de IA quanto ao conhecimento disponível, ao sinal de aprendizado e ao objeto produzido.

Paradigma	Conhecimento fornecido	Sinal de aprendizado	O que é produzido
Busca (A*, UCS)	Modelo completo	Nenhum	Plano ou caminho ótimo
Minimax	Modelo adversarial completo e função de utilidade	Função de utilidade	Melhor ação segundo a árvore de busca
MCTS	Regras do jogo / simulador	Resultados de simulações	Estimativas de valor das ações
Aprendizado por reforço	Nenhum modelo	Recompensa escalar	Política de ação
AM supervisionado	Nenhum modelo	Exemplos rotulados	Função entrada-saída
AM não supervisionado	Nenhum modelo	Nenhum rótulo	Estrutura nos dados

1.5 Quando usar aprendizado de máquina?

O AM é a ferramenta certa quando:

- O fenômeno é complexo demais para ser descrito por regras explícitas: reconhecer faces, traduzir idiomas e entender fala são tarefas que qualquer criança executa com facilidade, mas para as quais ninguém sabe escrever um conjunto completo de regras do tipo “*se a imagem tem estas bordas neste ângulo, então é um rosto*”. O espaço de variações possíveis é vasto demais, as exceções são incontáveis e as regras relevantes dependem de contexto de formas que resistem à formalização manual. Nesses casos, faz mais sentido deixar que o algoritmo infira as regularidades a partir de exemplos do que tentar enumerá-las a priori.
- Os dados estão disponíveis: há exemplos suficientes do comportamento desejado.
- O fenômeno é suficientemente estável para que a hipótese de aprendizagem seja plausível.

O AM não é a ferramenta certa quando:

- O problema tem uma solução exata eficiente: um algoritmo de busca ou de programação dinâmica pode ser mais apropriado.
- Os dados disponíveis são insuficientes, tendenciosos ou de má qualidade.
- A interpretabilidade é crítica e os dados não justificam modelos opacos.

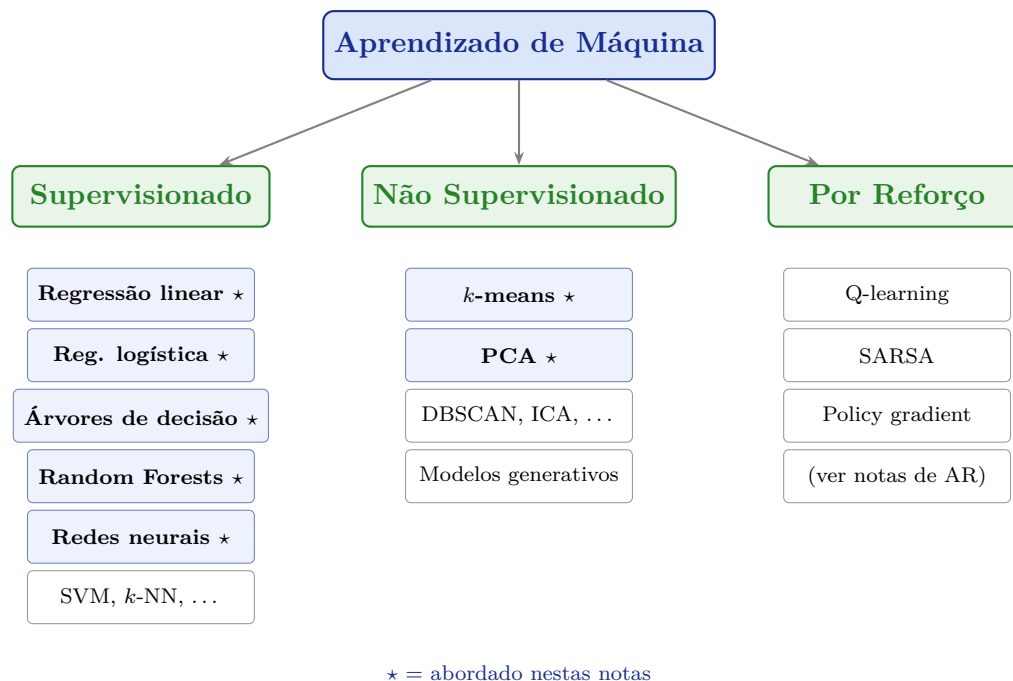


Figura 1: Taxonomia parcial do aprendizado de máquina. O paradigma de aprendizado por reforço, abordado nas notas anteriores, é aqui posicionado em relação aos demais. Os itens marcados com ★ são cobertos nesta leitura.

Três perguntas antes de usar AM

Antes de aplicar AM a um problema, pergunte:

1. Há dados suficientes e representativos do fenômeno?
2. Existe um padrão estável que pode ser aprendido?
3. O custo dos erros do modelo é aceitável no domínio?

Se a resposta a qualquer uma dessas perguntas for negativa, o problema talvez precise de outra abordagem: regras explícitas, modelagem causal, simulação, coleta de dados melhor ou intervenção humana.

Micro-checkpoint

Para cada situação abaixo, identifique o paradigma mais adequado (busca, AR, AM supervisionado, AM não supervisionado) e justifique: (a) filtrar e-mails de spam com base em exemplos rotulados; (b) ensinar um robô a andar em terrenos novos por tentativa e erro; (c) encontrar o caminho mais curto em um mapa; (d) agrupar clientes por padrão de compras, sem rótulos.

Pergunta 1

A hipótese de aprendizagem afirma que o passado é informativamente semelhante ao futuro. Descreva um cenário real em que essa hipótese claramente não é satisfeita e explique como isso pode comprometer um sistema de AM treinado com dados históricos. O que um praticante poderia fazer para detectar ou mitigar esse problema?

2 O Framework de Aprendizado Supervisionado

2.1 Dados, tarefas e modelos

No aprendizado supervisionado (AS), o ponto de partida é um *conjunto de dados de treinamento*:

$$\mathcal{D} = \{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(n)}, y^{(n)})\}$$

onde cada $\mathbf{x}^{(i)} \in \mathbb{R}^d$ é um *vetor de características (features)* e $y^{(i)}$ é o *rótulo* ou *alvo* correspondente. O índice i percorre os n exemplos de treinamento, e d é o número de características.

O objetivo do aprendizado supervisionado é encontrar uma função $h : \mathbb{R}^d \rightarrow \mathcal{Y}$, chamada *hipótese*, que mapeie corretamente entradas em saídas, tanto nos dados de treinamento quanto em entradas *nunca vistas antes*.

O tipo de saída $\mathcal{Y}$ define a natureza da tarefa:

- *Classificação*: $\mathcal{Y}$ é um conjunto finito de classes. Exemplo: $\mathcal{Y} = \{\text{spam}, \text{não-spam}\}$.
- *Regressão*: $\mathcal{Y} = \mathbb{R}$, um valor numérico contínuo. Exemplo: preço de um imóvel em reais.

Exemplo motivador: classificação de e-mails

Ao longo desta seção, usaremos um problema de classificação de spam como exemplo contínuo. Cada e-mail é representado por um vetor $\mathbf{x}$ cujas componentes capturam propriedades do e-mail: frequência de certas palavras (“grátis”, “promoção”, “clique”), comprimento do assunto, presença de links externos, entre outras. O rótulo $y \in \{0, 1\}$ indica se o e-mail é spam ($y = 1$) ou legítimo ($y = 0$). O objetivo é aprender uma função h que, dado um e-mail novo, preveja corretamente seu rótulo.

2.2 O espaço de hipóteses

Até aqui, falamos em aprender uma função h . Mas existem infinitas funções que poderiam ajustar os dados de alguma forma. Um algoritmo de AM não procura em todas elas: ele procura dentro de uma família restrita de funções. Essa família é o *espaço de hipóteses*. Assim, escolher um modelo é escolher uma forma de limitar o que o algoritmo pode aprender.

Formalmente, o espaço de hipóteses $\mathcal{H}$ é um conjunto de funções parametrizadas por um vetor de parâmetros $\boldsymbol{\theta}$:

$$\mathcal{H} = \{h_{\boldsymbol{\theta}} : \mathbb{R}^d \rightarrow \mathcal{Y} \mid \boldsymbol{\theta} \in \Theta\}$$

A escolha de $\mathcal{H}$ é uma decisão de projeto fundamental. Ela determina:

- *Capacidade expressiva*: que funções h podem ser representadas em $\mathcal{H}$? Uma família restrita (e.g., funções lineares) pode ser insuficiente para capturar padrões complexos. Uma família muito rica pode “memorizar” os dados sem generalizar.
- *Eficiência computacional*: o problema de encontrar o melhor h em $\mathcal{H}$ é tratável?
- *Interpretabilidade*: os parâmetros $\boldsymbol{\theta}$ têm significado compreensível para humanos?

Diferentes classes de modelos correspondem a diferentes escolhas de $\mathcal{H}$: funções lineares, árvores de decisão, redes neurais, etc.

Não existe almoço grátis (*no free lunch*)

O teorema “no free lunch” estabelece que, em geral, não existe um algoritmo de AM universalmente melhor que todos os outros. Todo algoritmo faz suposições implícitas sobre a estrutura do problema (o que é chamado de *viés indutivo* (*inductive bias*) do algoritmo). Um algoritmo eficiente para um tipo de problema pode ser ineficiente para outro.

Na prática, isso significa que a escolha do modelo (i.e., de $\mathcal{H}$) deve ser informada pelo conhecimento do domínio e validada empiricamente. Não existe o modelo “sempre melhor”; existe o modelo mais adequado para o problema e os dados disponíveis.

2.3 Função de perda

Depois de escolher uma família de funções, ainda falta decidir qual hipótese dentro dessa família é a melhor. Para isso, precisamos transformar a ideia intuitiva de “errar pouco” em uma quantidade numérica. Essa quantidade é a *função de perda* $\ell(h(\mathbf{x}), y)$, que mede o custo de prever $h(\mathbf{x})$ quando o valor verdadeiro é y .

Diferentes tarefas usam diferentes funções de perda:

- *Erro quadrático* (regressão): $\ell(\hat{y}, y) = (\hat{y} - y)^2$. Penaliza erros grandes de forma quadrática.
- *Erro absoluto* (regressão): $\ell(\hat{y}, y) = |\hat{y} - y|$. Mais robusto a valores atípicos.
- *Entropia cruzada binária* (classificação): $\ell(\hat{p}, y) = -y \log \hat{p} - (1 - y) \log(1 - \hat{p})$, onde $\hat{p}$ é a probabilidade prevista para a classe positiva. É a perda padrão para classificação logística.
- *Perda 0-1* (classificação): $\ell(\hat{y}, y) = \mathbf{1}[\hat{y} \neq y]$. Conta simplesmente se a previsão foi errada. Simples de interpretar, mas não diferenciável. Por isso, não é adequada para otimização direta por gradiente descendente; na prática, usa-se uma perda substituta diferenciável, como a entropia cruzada.

O *risco empírico* (ou custo de treinamento) é a média da perda sobre os dados de treinamento:

$$J(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \ell(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}), y^{(i)})$$

O objetivo do algoritmo de treinamento é encontrar $\boldsymbol{\theta}^*$ que minimize $J(\boldsymbol{\theta})$:

$$\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$$

Conexão com Q-learning

O leitor familiarizado com as notas de aprendizado por reforço reconhecerá aqui a mesma estrutura da atualização de pesos no Q-learning aproximado:

$$w_i \leftarrow w_i + \alpha \cdot \delta \cdot f_i(s, a)$$

onde δ é o erro temporal (diferença entre o alvo e a estimativa atual). Isso é equivalente a um passo de gradiente descendente sobre uma função perda quadrática entre o alvo de Bellman e a estimativa $Q(s, a) = \mathbf{w} \cdot \mathbf{f}(s, a)$. A única diferença é que, no AR, o alvo também depende dos pesos $\mathbf{w}$ (via $\max_{a'} Q(s', a')$), o que introduz instabilidade adicional.

2.4 O problema fundamental: generalização

A minimização do risco empírico $J(\theta)$ resolve um problema de otimização bem definido. Mas o objetivo real do AM não é desempenho nos dados de treinamento; é desempenho em dados *novos*, ainda não vistos. Essa capacidade é chamada de *generalização*.

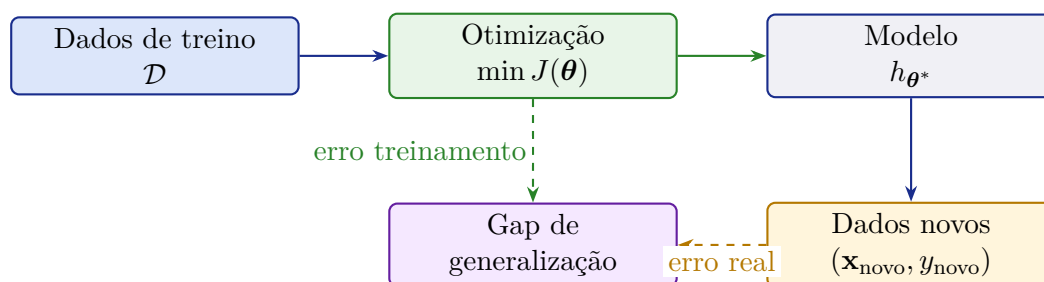
Se o objetivo fosse apenas reduzir $J(\theta)$, bastaria escolher uma classe de modelos suficientemente rica e memorizar o conjunto de treino. O problema é que memorizar não é aprender. Aprender exige transferir regularidades para casos novos.

Formalmente, o que queremos minimizar é o *risco esperado* (ou erro de generalização):

$$R(\theta) = \mathbb{E}_{(\mathbf{x}, y) \sim p_{\text{dados}}} [\ell(h_{\theta}(\mathbf{x}), y)]$$

onde a expectativa é tomada sobre a distribuição verdadeira dos dados p_{dados} . Infelizmente, p_{dados} é desconhecida. Tudo o que temos é uma amostra finita $\mathcal{D}$.

A diferença entre $R(\theta)$ (o que queremos minimizar) e $J(\theta)$ (o que realmente minimizamos) é o coração de todo o AM supervisionado. A Figura 2 ilustra esse gap.



O objetivo real é minimizar o erro em dados novos, não nos dados de treino.

Figura 2: O gap de generalização. O algoritmo de aprendizado minimiza o erro nos dados de treino, mas o que importa é o desempenho em dados novos. A diferença entre os dois é o gap de generalização, e reduzi-la é o problema central do aprendizado supervisionado.

Memorizar não é generalizar

Um modelo suficientemente flexível pode atingir erro quase zero no treino simplesmente memorizando os exemplos. Isso não significa que ele aprendeu a regularidade do fenômeno. A evidência de aprendizagem vem do desempenho em dados que não participaram do ajuste dos parâmetros.

2.5 Divisão dos dados: treino, validação e teste

Como p_{dados} é desconhecida, a prática padrão é estimar o erro de generalização usando dados que *não foram vistos durante o treinamento*. Para isso, o conjunto de dados disponível é dividido em três partes:

1. *Conjunto de treinamento*: usado para ajustar os parâmetros θ do modelo.
2. *Conjunto de validação*: usado para comparar diferentes modelos e ajustar *hiperparâmetros* (parâmetros que controlam o próprio processo de aprendizado, como a complexidade do modelo ou a taxa de aprendizado).
3. *Conjunto de teste*: usado *uma única vez* ao final, para estimar o desempenho real do modelo escolhido em dados novos.

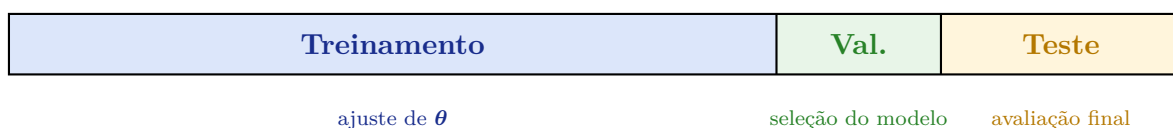


Figura 3: Divisão típica do conjunto de dados em treino, validação e teste. A região maior (treino) é usada para ajustar os parâmetros. A validação fica separada porque participa das escolhas de projeto: seleção de modelo e ajuste de hiperparâmetros. O teste aparece isolado porque não deve influenciar nenhuma decisão de desenvolvimento; é consultado apenas na avaliação final.

O erro mais comum em AM: vazamento de dados

Um erro gravíssimo em AM é usar informações do conjunto de teste durante o desenvolvimento do modelo, seja para selecionar características, ajustar hiperparâmetros ou comparar alternativas. Esse erro é chamado de *vazamento de dados* (*data leakage*) e produz estimativas excessivamente otimistas do desempenho real.

A regra de ouro é: *o conjunto de teste deve ser tratado como se não existisse até o momento da avaliação final*. Qualquer decisão de desenvolvimento deve se basear apenas nos conjuntos de treino e validação.

Micro-checkpoint

Explique com suas palavras a diferença entre um *parâmetro* e um *hiperparâmetro*. Dê um exemplo de cada. Por que o conjunto de validação é necessário, além do conjunto de teste?

Pergunta 2

Um estudante treina um classificador de spam com 10.000 e-mails. Ele obtém 98% de acurácia nos dados de treinamento. Quando coloca o modelo em produção, o desempenho cai para 72%. Identifique pelo menos três causas possíveis para essa discrepância. Para cada causa, proponha uma forma de detectá-la antes da implantação.

3 Modelos Lineares

3.1 Regressão linear

O modelo mais simples de AM supervisionado é a *regressão linear*. Dado um vetor de características $\mathbf{x} = [x_1, x_2, \dots, x_d]^\top$, o modelo prevê:

$$h_{\mathbf{w},b}(\mathbf{x}) = w_1x_1 + w_2x_2 + \dots + w_dx_d + b = \mathbf{w}^\top \mathbf{x} + b$$

onde $\mathbf{w} = [w_1, \dots, w_d]^\top$ é o vetor de pesos e b é o *viés* (*bias*), um parâmetro escalar que desloca a previsão.

Convenção: absorvendo o viés

É conveniente absorver b no vetor de pesos adicionando uma componente constante $x_0 = 1$ ao vetor de características: $\mathbf{x} \leftarrow [1, x_1, x_2, \dots, x_d]^\top$ e $\mathbf{w} \leftarrow [b, w_1, \dots, w_d]^\top$. Com isso, $h_{\mathbf{w}}(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}$, e toda análise é feita em termos de um único vetor de parâmetros $\mathbf{w} \in \mathbb{R}^{d+1}$. Essa convenção é padrão na literatura e simplifica a notação.

Para regressão linear, a função de perda padrão é o *erro quadrático médio* (*mean squared error*, MSE):

$$J(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (h_{\mathbf{w}}(\mathbf{x}^{(i)}) - y^{(i)})^2 = \frac{1}{n} \sum_{i=1}^n (\mathbf{w}^\top \mathbf{x}^{(i)} - y^{(i)})^2$$

Existe uma solução analítica fechada para o vetor ótimo de pesos, chamada *equação normal*:

$$\mathbf{w}^* = (X^\top X)^{-1} X^\top \mathbf{y}$$

onde X é a matriz de dados ($n \times (d+1)$, uma linha por exemplo) e $\mathbf{y}$ é o vetor de rótulos.

Quando a equação normal falha

A equação normal requer inverter a matriz $X^\top X$, de tamanho $(d+1) \times (d+1)$. Isso tem custo $O(d^3)$, o que a torna impraticável para d grande (e.g., $d = 10^6$ em problemas de processamento de linguagem natural). Além disso, $X^\top X$ pode ser singular (não invertível) se as características forem linearmente dependentes. O gradiente descendente, apresentado a seguir, contorna ambos os problemas.

3.1.1 Exemplo resolvido

Suponha um problema de regressão com apenas uma característica ($d = 1$): prever o preço de um imóvel (y , em mil reais) em função de sua área (x , em m²). Dados de treinamento:

Exemplo	x (área, m ²)	y (preço, R\$ mil)
1	50	200
2	100	350
3	150	500

Com $\mathbf{w} = [b, w_1]^\top$ e $x_0 = 1$, temos $h(x) = b + w_1x$.

Passo 1: verificar um candidato. Suponha $b = 50$ e $w_1 = 3$, ou seja, $h(x) = 50 + 3x$.

$$\begin{aligned}h(50) &= 50 + 3 \times 50 = 200 \quad (\text{erro} = 0) \\h(100) &= 50 + 3 \times 100 = 350 \quad (\text{erro} = 0) \\h(150) &= 50 + 3 \times 150 = 500 \quad (\text{erro} = 0)\end{aligned}$$

Neste caso, os dados são exatamente lineares, então $J(\mathbf{w}) = 0$ é atingível.

Passo 2: verificar um candidato ruim. Suponha $b = 0$ e $w_1 = 4$:

$$\begin{aligned}J(\mathbf{w}) &= \frac{1}{3}[(200 - 200)^2 + (400 - 350)^2 + (600 - 500)^2] \\&= \frac{1}{3}[0 + 2500 + 10000] = 4166,7\end{aligned}$$

O MSE positivo indica que este não é o modelo ótimo.

3.2 Gradiente descendente

Para minimizar $J(\mathbf{w})$, o algoritmo mais geral é o *gradiente descendente*. A ideia é simples: a partir de um ponto inicial $\mathbf{w}^{(0)}$, repetidamente mova-se na direção oposta ao gradiente de J , que é a direção de maior descida:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla_{\mathbf{w}} J(\mathbf{w})$$

onde $\alpha > 0$ é a *taxa de aprendizado* e $\nabla_{\mathbf{w}} J(\mathbf{w})$ é o gradiente de J em relação a $\mathbf{w}$.

Para regressão linear com MSE, o gradiente em relação ao peso w_j é:

$$\frac{\partial J}{\partial w_j} = \frac{2}{n} \sum_{i=1}^n (\mathbf{w}^\top \mathbf{x}^{(i)} - y^{(i)}) x_j^{(i)}$$

A regra de atualização para cada peso w_j é, portanto:

$$w_j \leftarrow w_j - \alpha \cdot \frac{2}{n} \sum_{i=1}^n \underbrace{(\mathbf{w}^\top \mathbf{x}^{(i)} - y^{(i)})}_{\text{erro na previsão}} x_j^{(i)}$$

Isso tem uma interpretação intuitiva: se o modelo está superestimando ($\hat{y} > y$) e a característica x_j foi positiva, então w_j foi muito alto e deve ser reduzido.

Escala das características

O gradiente descendente é sensível à escala das características. Se uma variável varia entre 0 e 10.000 e outra entre 0 e 1, os pesos correspondentes podem ser atualizados em ritmos muito diferentes, tornando a convergência lenta ou instável. Por isso, é comum padronizar ou normalizar as características antes do treinamento — por exemplo, subtraindo a média e dividindo pelo desvio padrão.

Analogia com Q-learning

Comparando a regra de atualização acima com a do Q-learning aproximado:

$$w_j \leftarrow w_j + \alpha \cdot \underbrace{\delta}_{\text{erro TD}} \cdot f_j(s, a)$$

a estrutura é idêntica: ajuste proporcional ao erro (TD ou de regressão) vezes o valor da característica. No Q-learning, $\delta = [r + \gamma \max_{a'} Q(s', a')] - Q(s, a)$; na regressão linear, $\delta = y^{(i)} - \hat{y}^{(i)}$. Ambos minimizam um erro quadrático entre uma previsão e um alvo.

Variantes de gradiente descendente. Na prática, calcular o gradiente sobre *todos* os n exemplos a cada passo pode ser custoso. Há três variantes:

- *Gradiente descendente em lote (batch gradient descent)*: usa todos os n exemplos. Atualização precisa, mas custosa para n grande.
- *Gradiente descendente estocástico (stochastic gradient descent, SGD)*: usa um único exemplo por passo, escolhido aleatoriamente. Atualização ruidosa, mas muito rápida. Convergência instável, mas frequentemente suficiente.
- *Gradiente descendente em mini-lote (mini-batch gradient descent)*: usa um subconjunto de b exemplos por passo (b típico: 32–256). Compromisso prático. É o padrão em aprendizado profundo.

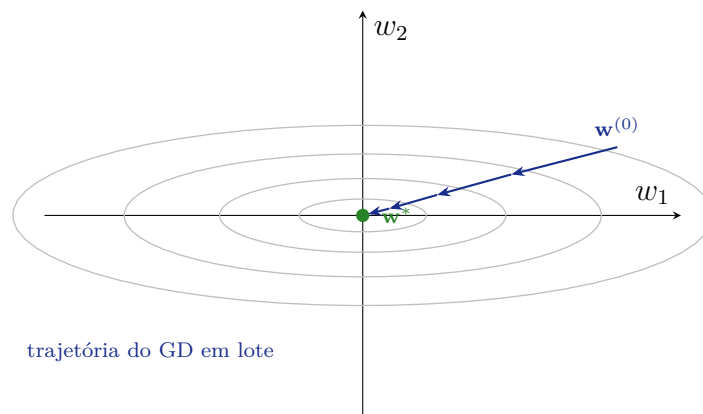


Figura 4: Trajetória do gradiente descendente em lote num espaço bidimensional de parâmetros. As curvas de nível representam o custo $J(\mathbf{w})$; a trajetória parte de $\mathbf{w}^{(0)}$ e converge para o mínimo $\mathbf{w}^*$, movendo-se sempre na direção de maior descida.

3.2.1 Exemplo resolvido: um passo de gradiente descendente

Continuando o exemplo anterior com $d = 1$: $h(x) = b + w_1x$. Suponha que os pesos atuais sejam $b = 0$, $w_1 = 4$, e $\alpha = 0,0001$.

Passo 1: calcular os erros.

$$\begin{aligned}e_1 &= h(50) - 200 = 200 - 200 = 0 \\e_2 &= h(100) - 350 = 400 - 350 = +50 \\e_3 &= h(150) - 500 = 600 - 500 = +100\end{aligned}$$

Passo 2: calcular os gradientes.

$$\frac{\partial J}{\partial w_1} = \frac{2}{3}(0 \cdot 50 + 50 \cdot 100 + 100 \cdot 150) = \frac{2}{3}(0 + 5000 + 15000) = \frac{40000}{3} \approx 13333$$

$$\frac{\partial J}{\partial b} = \frac{2}{3}(0 + 50 + 100) = 100$$

Passo 3: atualizar os pesos.

$$w_1 \leftarrow 4 - 0,0001 \times 13333 \approx 4 - 1,33 = 2,67$$

$$b \leftarrow 0 - 0,0001 \times 100 = -0,01$$

Interpretação: w_1 desceu de 4 para 2,67 (em direção ao valor ótimo 3), pois o modelo estava superestimando os preços de imóveis maiores. O processo se repete até convergência.

3.3 Regressão logística e classificação binária

Para classificação binária ($y \in \{0, 1\}$), um modelo linear simples $h(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}$ não é adequado como probabilidade: ele pode produzir valores negativos ou maiores que 1. A *regressão logística* aplica a função *sigmoide* $\sigma(z) = \frac{1}{1+e^{-z}}$ à saída linear para forçar o resultado ao intervalo (0, 1):

$$h_{\mathbf{w}}(\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^\top \mathbf{x}}}$$

O valor $h_{\mathbf{w}}(\mathbf{x}) \in (0, 1)$ é interpretado como a probabilidade de $y = 1$ dado $\mathbf{x}$. A decisão final é obtida por limiarização:

$$\hat{y} = \begin{cases} 1 & \text{se } h_{\mathbf{w}}(\mathbf{x}) \geq 0,5 \\ 0 & \text{caso contrário} \end{cases}$$

O limiar 0,5 corresponde a $\mathbf{w}^\top \mathbf{x} = 0$, o *hiperplano de decisão* que separa as duas classes.

A função de perda adequada para regressão logística é a *entropia cruzada binária*:

$$J(\mathbf{w}) = -\frac{1}{n} \sum_{i=1}^n [y^{(i)} \log h(\mathbf{x}^{(i)}) + (1 - y^{(i)}) \log(1 - h(\mathbf{x}^{(i)}))]$$

Essa função penaliza fortemente previsões confiantes e erradas. Se o modelo prevê $h(\mathbf{x}) = 0,99$ (muito confiante em $y = 1$) mas o rótulo verdadeiro é $y = 0$, a perda é $-\log(0,01) \approx 4,6$, que é muito alta. O gradiente descendente pode ser aplicado diretamente para minimizar esta perda.

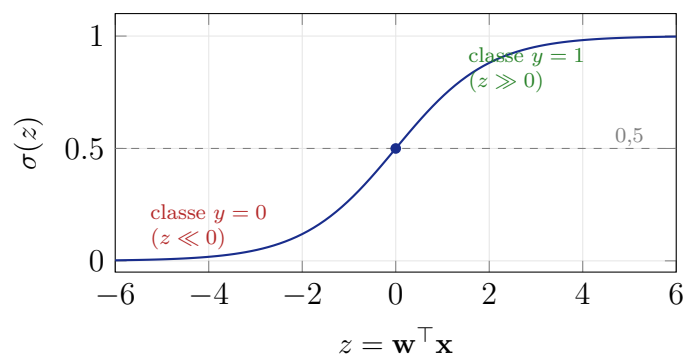


Figura 5: A função sigmoide $\sigma(z)$, usada na regressão logística. Ela comprime qualquer valor real $z = \mathbf{w}^\top \mathbf{x}$ para o intervalo $(0, 1)$, produzindo uma estimativa de probabilidade. Para $z \ll 0$, a saída se aproxima de 0 (classe negativa); para $z \gg 0$, se aproxima de 1 (classe positiva). O limiar de decisão padrão é 0,5, que corresponde a $z = 0$.

Regressão logística não é regressão no sentido usual

O nome “regressão logística” é historicamente enganoso: trata-se de um modelo de *classificação*, não de regressão. A confusão ocorre porque o modelo estima uma probabilidade contínua (regressão) e depois limiariza para produzir uma classificação binária. A denominação persiste por razões históricas.

3.4 Engenharia de características

Modelos lineares têm limitações óbvias: eles só conseguem representar fronteiras de decisão lineares (hiperplanos). No entanto, a maioria dos problemas reais envolve relações não lineares. Como lidar com isso mantendo a tratabilidade dos modelos lineares?

A resposta é a *engenharia de características*: transformar as variáveis originais em um novo espaço de representação onde as relações sejam (mais próximas de) lineares. Algumas transformações comuns:

- *Características polinomiais*: adicionar x_1^2 , x_1x_2 , x_2^2 , etc. Permite capturar relações polinomiais.
- *Logaritmo*: $\log x_j$ para variáveis com distribuições muito assimétricas (e.g., rendas, preços).
- *Interações*: $x_j \cdot x_k$ captura relações de multiplicação entre características.
- *Binarização*: converter variáveis categóricas em variáveis binárias (*one-hot encoding*).

Engenharia de características e Q-learning

Os leitores das notas de AR já encontraram engenharia de características no Q-learning com aproximação linear: as funções $f_i(s, a)$ são exatamente características projetadas manualmente para capturar aspectos relevantes do par estado-ação. O princípio é o mesmo: um modelo linear no espaço de características pode representar relações não lineares no espaço original.

O risco de características polinomiais: overfitting

Adicionar características polinomiais de grau alto permite que o modelo se ajuste perfeitamente aos dados de treinamento, interpolando por todos os pontos. Mas isso geralmente prejudica a generalização, pois o modelo “memoriza” os dados em vez de aprender a tendência subjacente. Esse fenômeno é chamado de *overfitting* e é estudado em detalhes na próxima seção.

Micro-checkpoint

Explique por que a regressão linear pura não consegue classificar pontos que não são linearmente separáveis. Descreva uma transformação de características que permitiria a um modelo linear separar os pontos $\{(1, 1), (-1, -1)\}$ (classe +) dos pontos $\{(1, -1), (-1, 1)\}$ (classe -). (*Dica:* observe que esses pontos são separáveis por uma fronteira do tipo $x_1 \cdot x_2 = 0$.)

Pergunta 3

Um modelo de regressão linear treinado para prever o consumo de energia elétrica de edifícios usa como características a área útil e o número de andares. Após treinamento, o modelo obtém $MSE = 250$ no conjunto de treino e $MSE = 1800$ no conjunto de teste. Identifique o problema e proponha três ações concretas (relacionadas aos dados, ao modelo ou ao processo de avaliação) para melhorar o desempenho de generalização.

4 Generalização: Viés, Variância e Regularização

4.1 Underfitting e overfitting

Toda a dificuldade do AM supervisionado pode ser resumida em um único dilema: um modelo muito simples não captura os padrões dos dados (*underfitting*); um modelo muito complexo captura até o ruído dos dados de treinamento e não generaliza (*overfitting*).

A Figura 7 mostra as curvas de erro típicas de treino e validação em função da complexidade do modelo.

4.2 O dilema viés-variância

O dilema viés-variância é a formalização matemática do problema de underfitting e overfitting. Para qualquer modelo e qualquer ponto de teste $\mathbf{x}$, o erro quadrático esperado pode ser decomposto como:

$$\mathbb{E}[(h(\mathbf{x}) - y)^2] = \underbrace{\text{Viés}^2(h(\mathbf{x}))}_{\text{erro sistemático}} + \underbrace{\text{Variância}(h(\mathbf{x}))}_{\text{sensibilidade ao dado}} + \underbrace{\sigma_{\text{ruído}}^2}_{\text{irreduzível}}$$

onde a expectativa é tomada sobre diferentes conjuntos de treinamento amostrados da mesma distribuição.

- *Viés*: mede o quanto as previsões do modelo são sistematicamente erradas, independentemente dos dados de treinamento. Um modelo linear aplicado a um problema intrinsecamente quadrático tem alto viés.

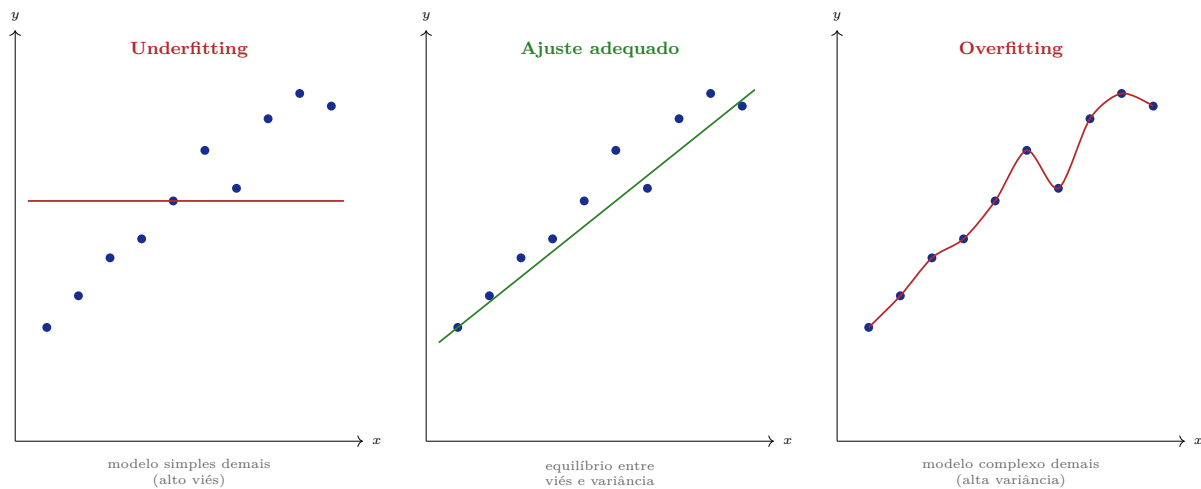


Figura 6: Três casos de ajuste em regressão. À esquerda, o modelo é simples demais (linha plana) e não captura a tendência crescente dos dados: underfitting. No centro, o modelo é linear e captura a tendência sem se prender ao ruído: ajuste adequado. À direita, o modelo passa exatamente por todos os pontos mas oscilará muito em dados novos: overfitting.

- *Variância*: mede o quanto as previsões do modelo mudam quando o conjunto de treinamento muda. Um modelo polinomial de grau alto, treinado em amostras diferentes da mesma distribuição, produz previsões muito diferentes (alta variância).
- *Ruído irreduzível*: a variabilidade inerente dos dados ($y = f(\mathbf{x}) + \varepsilon$), que nenhum modelo pode eliminar.

Viés-variância no AR

O dilema viés-variância já apareceu implicitamente nas notas de AR, na comparação entre Monte Carlo e TD. MC tem baixo viés (usa retornos reais) mas alta variância (o retorno depende de toda a sequência de recompensas ruidosas). TD tem viés introduzido pelo bootstrap (o alvo usa estimativas), mas menor variância (o alvo é mais estável). A escolha entre MC e TD é, portanto, uma escolha no espaço viés-variância.

4.3 Regularização

A regularização é a técnica principal para controlar overfitting sem aumentar o conjunto de dados. A ideia é adicionar um *termo de penalidade* à função de custo que favorece modelos mais simples (com pesos menores):

$$J_{\text{reg}}(\mathbf{w}) = J(\mathbf{w}) + \lambda \cdot \Omega(\mathbf{w})$$

onde $\lambda > 0$ é o *hiperparâmetro de regularização* que controla o equilíbrio entre ajuste aos dados e simplicidade do modelo. As duas formas mais comuns são:

Regularização ℓ_2 (Ridge):

$$\Omega(\mathbf{w}) = \|\mathbf{w}\|_2^2 = \sum_j w_j^2$$

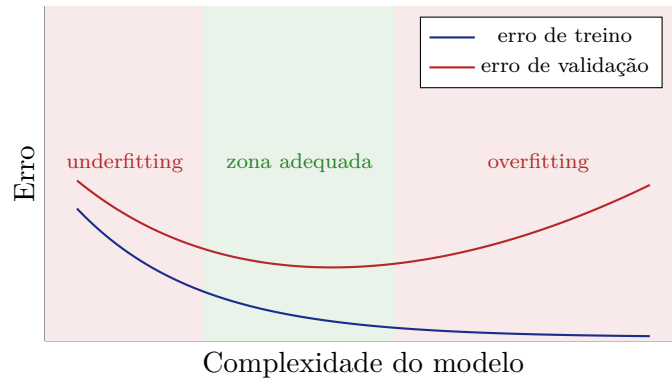


Figura 7: Curvas de erro em função da complexidade do modelo. O erro de treino decresce monotonicamente e se aproxima de zero (o modelo memoriza os dados). O erro de validação tem formato de U: decresce até um ponto ótimo, depois sobe (overfitting). A complexidade ótima é aquela que minimiza o erro de validação.

Penaliza pesos grandes, forçando-os a permanecer próximos de zero. Todos os pesos são reduzidos, mas raramente chegam a zero exatamente.

Regularização ℓ_1 (Lasso):

$$\Omega(\mathbf{w}) = \|\mathbf{w}\|_1 = \sum_j |w_j|$$

Produce soluções *esparsas*: muitos pesos tornam-se exatamente zero, efetivamente selecionando um subconjunto de características.

Uma forma intuitiva de visualizar a regularização é imaginar que o otimizador procura a menor perda, mas dentro de uma região permitida para os pesos. A forma dessa região explica por que ℓ_2 apenas encolhe pesos, enquanto ℓ_1 tende a zerar alguns deles, como mostra a Figura 8.

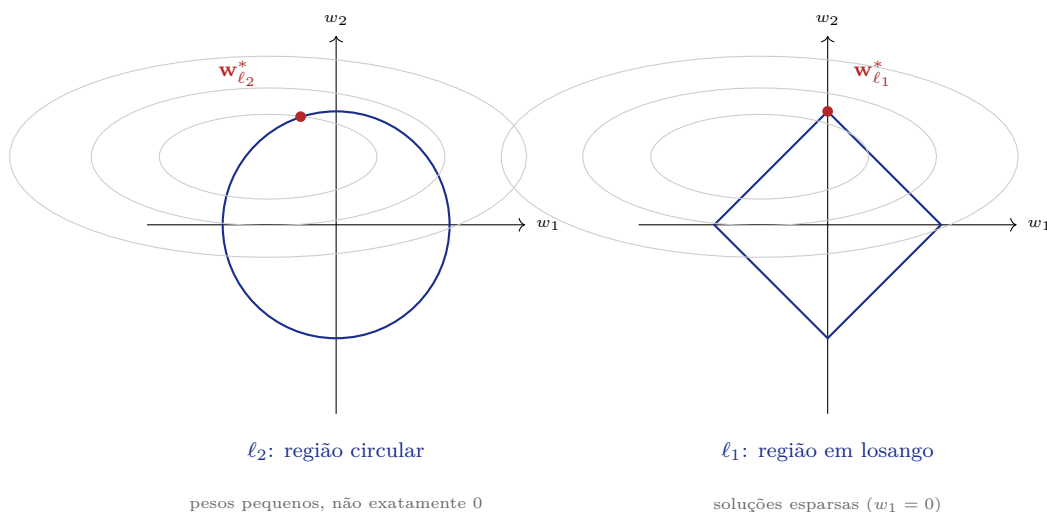


Figura 8: Intuição geométrica da regularização ℓ_2 e ℓ_1 . A solução regularizada é o ponto de menor custo (curvas de nível) que toca a região permitida para os pesos. Na ℓ_2 (bola circular), a solução se desloca para um ponto arbitrário da borda. Na ℓ_1 (losango), as soluções tendem a tocar os vértices, onde uma das componentes é zero, daí a esparsidade.

Regularização como prior bayesiano

A regularização ℓ_2 é equivalente a adicionar um *prior* gaussiano centrado em zero sobre os pesos: assume-se que, na ausência de evidência contrária, os pesos devem ser pequenos. A regularização ℓ_1 corresponde a um *prior* de Laplace, que tem caudas mais pesadas e favorece esparsidade. Embora a derivação bayesiana seja formal demais para este nível, a intuição é útil: regularizar é incorporar uma crença a priori de que modelos simples são mais plausíveis.

4.3.1 Exemplo resolvido

Suponha um modelo linear com dois pesos $w_1 = 3,0$ e $w_2 = 0,1$, e custo de treinamento $J = 1,5$, com $\lambda = 0,5$.

Custo regularizado ℓ_2 :

$$J_{\ell_2} = 1,5 + 0,5 \times (3,0^2 + 0,1^2) = 1,5 + 0,5 \times 9,01 = 1,5 + 4,505 = 6,005$$

Custo regularizado ℓ_1 :

$$J_{\ell_1} = 1,5 + 0,5 \times (|3,0| + |0,1|) = 1,5 + 0,5 \times 3,1 = 1,5 + 1,55 = 3,05$$

Interpretação: a regularização penaliza $w_1 = 3,0$ muito mais do que $w_2 = 0,1$. O otimizador é incentivado a reduzir w_1 , a menos que mantê-lo grande seja fortemente justificado pelos dados. Isso previne que um único peso domine o modelo e cause overfitting.

4.4 Validação cruzada

Quando o conjunto de dados é pequeno, reservar 15–20% para validação pode desperdiçar dados preciosos de treinamento. A *validação cruzada k-fold* resolve esse problema:

1. Divida o conjunto de treino em k partes (*folds*) de tamanho igual.
2. Para $i = 1, \dots, k$: treine o modelo em todos os folds exceto o i -ésimo; avalie no fold i .
3. O erro de validação estimado é a média dos k erros obtidos.

Validação cruzada não substitui o conjunto de teste

A validação cruzada substitui a separação treino/validação durante a escolha de modelos e hiperparâmetros, permitindo usar os dados de forma mais eficiente. No entanto, ela não substitui o conjunto de teste. O teste continua reservado para a avaliação final, depois que todas as escolhas de modelo e hiperparâmetros foram feitas. Aplicar validação cruzada ao conjunto de teste é um erro equivalente ao vazamento de dados.

4.5 Métricas de avaliação

A escolha da métrica de avaliação depende da tarefa e dos requisitos do domínio.

iter 5	treino	treino	treino	treino	val
iter 4	treino	treino	treino	val	treino
iter 3	treino	treino	val	treino	treino
iter 2	treino	val	treino	treino	treino
iter 1	val	treino	treino	treino	treino

5-fold: cada iteração usa um fold diferente como validação. Erro final = média dos 5 erros.

Figura 9: Validação cruzada 5-fold. A cada iteração, um fold diferente é usado para validação e os restantes para treinamento. O erro estimado é a média dos 5 erros de validação, usando todos os dados para treino ao longo do processo. O conjunto de teste permanece completamente separado e não participa da validação cruzada.

Regressão.

- MSE (Erro Quadrático Médio): $\frac{1}{n} \sum (y_i - \hat{y}_i)^2$. Penaliza erros grandes.
- $RMSE$: $\sqrt{MSE}$. Na mesma unidade da variável-alvo.
- MAE (Erro Absoluto Médio): $\frac{1}{n} \sum |y_i - \hat{y}_i|$. Mais robusto a outliers.
- R^2 (coeficiente de determinação): fração da variância explicada pelo modelo. $R^2 = 1$ indica ajuste perfeito; $R^2 = 0$, não melhor que prever a média.

Classificação. A métrica mais comum é a *acurácia*: fração de exemplos classificados corretamente. Mas ela pode ser enganosa em dados desbalanceados. Se 95% dos exemplos são da classe 0, um modelo que sempre prevê 0 tem acurácia 95% mas é inútil.

Para analisar melhor o desempenho, usamos a *matriz de confusão* (Tabela 2) e métricas derivadas. A distinção entre falso positivo e falso negativo é essencial porque os dois tipos de erro podem ter custos muito diferentes dependendo do domínio.

Tabela 2: Matriz de confusão para classificação binária.

		Previsto	
		Positivo	Negativo
Real	Positivo	VP (Verdadeiro Positivo)	FN (Falso Negativo)
	Negativo	FP (Falso Positivo)	VN (Verdadeiro Negativo)

- **Precisão**: $\frac{VP}{VP+FP}$ dos que foram classificados como positivos, quantos realmente são?
- **Revocação** (*recall*): $\frac{VP}{VP+FN}$ dos que são positivos, quantos o modelo encontrou?
- **F1-score**: média harmônica de precisão e revocação: $\frac{2 \cdot \text{Precisão} \cdot \text{Revocação}}{\text{Precisão} + \text{Revocação}}$.

Precisão *versus* revocação: o dilema do médico

Em diagnóstico médico, um falso negativo (não detectar uma doença grave) é geralmente muito pior do que um falso positivo (suspeitar de uma doença que não existe). Nesse contexto, maximizar a revocação é prioritário, mesmo que isso reduza a precisão. Já em filtragem de spam, um falso positivo (deletar um e-mail legítimo) pode ser pior do que um falso negativo (deixar passar um spam). A escolha da métrica deve refletir as consequências reais dos erros no domínio.

Micro-checkpoint

Um modelo de AM para detecção de fraude em transações bancárias tem 99,8% de acurácia, mas a equipe de segurança está insatisfeita. Explique por que a acurácia pode ser uma métrica enganosa neste contexto. Qual combinação de métricas seria mais informativa?

Pergunta 4

Um engenheiro de ML treina três versões de um modelo de regressão usando características polinomiais de graus 1, 5 e 15. O modelo de grau 1 obtém MSE de treino = 120 e MSE de validação = 125. O de grau 5 obtém MSE de treino = 45 e MSE de validação = 52. O de grau 15 obtém MSE de treino = 2 e MSE de validação = 380. (a) Qual modelo apresenta underfitting? Qual apresenta overfitting? (b) Qual você selecionaria para implantação e por quê? (c) Se você quisesse melhorar o modelo de grau 15 sem reduzir o grau, que estratégia usaria?

5 Árvores de Decisão e Métodos Ensemble

5.1 Árvores de decisão

Modelos lineares representam fronteiras de decisão como hiperplanos. As *árvores de decisão* adotam uma abordagem radicalmente diferente: particionam o espaço de entradas em regiões hierárquicas por meio de uma sequência de testes simples (geralmente do tipo $x_j \leq \tau$), e fazem uma previsão constante em cada região folha.

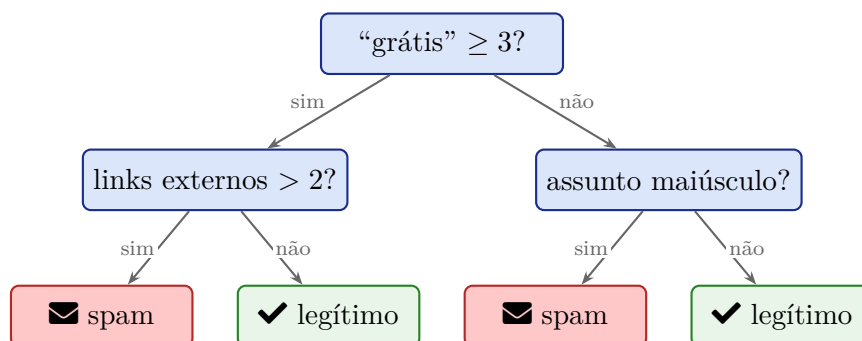


Figura 10: Exemplo de árvore de decisão para classificação de spam. Cada nó interno testa uma característica; cada folha produz uma previsão. A interpretabilidade é uma vantagem central: é fácil explicar por que um e-mail foi classificado como spam.

A principal vantagem das árvores de decisão é a *interpretabilidade*: cada decisão pode

ser rastreada por um caminho explícito na árvore. A principal desvantagem é que, sem controle de profundidade, árvores profundas tendem a sofrer overfitting severo.

5.2 Entropia e ganho de informação

O problema central na construção de uma árvore é: *qual atributo usar em cada nó de divisão?* O critério mais comum é o *ganho de informação*, baseado no conceito de *entropia* da teoria da informação.

É importante notar que a construção da árvore é *gulosa*: em cada nó, escolhe-se a melhor divisão local segundo o critério de ganho de informação. Isso não garante a árvore globalmente ótima, mas produz um algoritmo eficiente e interpretável.

Para um conjunto de exemplos S com proporção p_k da classe k :

$$H(S) = - \sum_k p_k \log_2 p_k$$

A entropia mede a impureza (ou desordem) do conjunto. Ela é máxima quando as classes estão igualmente distribuídas ($H = \log_2 K$ para K classes) e mínima ($H = 0$) quando todos os exemplos pertencem à mesma classe.

O *ganho de informação* de um atributo A com valores possíveis $\{v_1, v_2, \dots\}$ é:

$$\text{IG}(S, A) = H(S) - \sum_v \frac{|S_v|}{|S|} H(S_v)$$

onde S_v é o subconjunto de S em que $A = v$. O atributo escolhido para divisão é aquele que maximiza $\text{IG}(S, A)$.

5.2.1 Exemplo resolvido

Considere um conjunto de 10 e-mails: 6 spam (+) e 4 legítimos (-). Dois atributos candidatos para divisão:

- Atributo A (“grátis” ≥ 3): divide em $S_{\text{sim}} = \{5+, 1-\}$ e $S_{\text{não}} = \{1+, 3-\}$.
- Atributo B (links > 2): divide em $S_{\text{sim}} = \{3+, 3-\}$ e $S_{\text{não}} = \{3+, 1-\}$.

Passo 1: calcular a entropia do conjunto raiz.

$$H(S) = -\frac{6}{10} \log_2 \frac{6}{10} - \frac{4}{10} \log_2 \frac{4}{10} = -0,6 \times (-0,737) - 0,4 \times (-1,322) = 0,442 + 0,529 = 0,971$$

Passo 2: calcular IG do Atributo A.

$$H(S_{\text{sim}}) = -\frac{5}{6} \log_2 \frac{5}{6} - \frac{1}{6} \log_2 \frac{1}{6} \approx 0,650$$

$$H(S_{\text{não}}) = -\frac{1}{4} \log_2 \frac{1}{4} - \frac{3}{4} \log_2 \frac{3}{4} \approx 0,811$$

$$\text{IG}(S, A) = 0,971 - \left(\frac{6}{10} \times 0,650 + \frac{4}{10} \times 0,811 \right) = 0,971 - (0,390 + 0,324) = 0,971 - 0,714 = 0,257$$

Passo 3: calcular IG do Atributo B.

$$H(S_{\text{sim}}) = -\frac{3}{6} \log_2 \frac{3}{6} - \frac{3}{6} \log_2 \frac{3}{6} = 1,0$$

$$H(S_{\text{não}}) = -\frac{3}{4} \log_2 \frac{3}{4} - \frac{1}{4} \log_2 \frac{1}{4} \approx 0,811$$

$$\text{IG}(S, B) = 0,971 - \left(\frac{6}{10} \times 1,0 + \frac{4}{10} \times 0,811 \right) = 0,971 - (0,600 + 0,324) = 0,971 - 0,924 = 0,047$$

Decisão: o Atributo A (IG = 0,257) é muito melhor que o B (IG = 0,047). O algoritmo escolhe A para dividir o nó raiz. Intuitivamente, A cria subconjuntos muito mais puros (mais homogêneos em termos de classe).

5.3 Controle de complexidade e poda

Uma árvore construída até esgotar os dados de treinamento pode ter profundidade muito grande e sofreria overfitting grave. Duas estratégias principais controlam isso:

- *Pré-poda (pre-pruning)*: parar a divisão cedo, quando o ganho de informação for menor que um limiar, ou quando o nó tiver menos que $m_{\min}$ exemplos.
- *Pós-poda (post-pruning)*: construir a árvore completa e depois podar ramos cujo desempenho de validação não melhora com sua remoção.

A instabilidade das árvores de decisão

Árvores de decisão são altamente sensíveis a pequenas perturbações nos dados de treinamento: remover ou adicionar poucos exemplos pode produzir árvores muito diferentes. Essa instabilidade (alta variância) é a principal motivação para os métodos ensemble apresentados a seguir. A ideia central é: se uma única árvore é instável, uma combinação de muitas árvores pode ser estável e precisa.

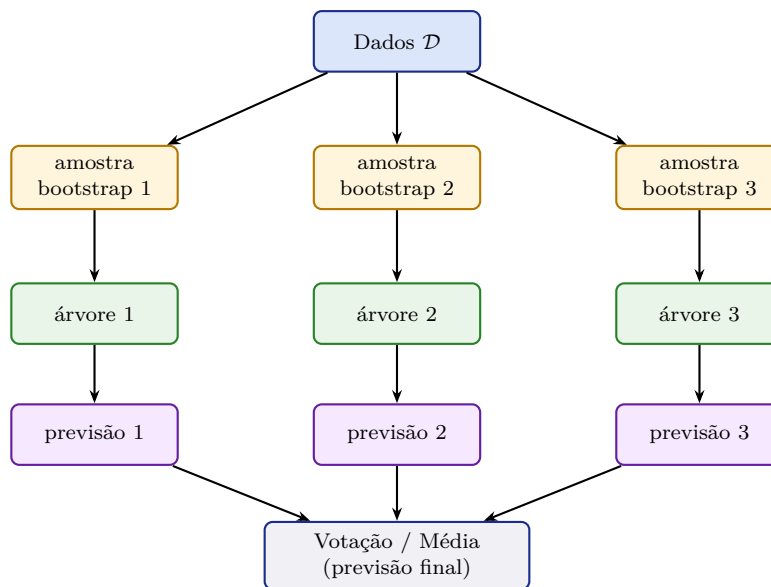
5.4 Métodos ensemble

Um *método ensemble* combina as previsões de múltiplos modelos (chamados de *modelos base* ou *aprendizes fracos*) para produzir uma previsão final mais robusta. A intuição é que modelos diferentes erram em lugares diferentes, e sua combinação cancela esses erros.

Bagging (*bootstrap aggregating*). O *bagging* reduz a variância treinando T modelos independentes em subconjuntos diferentes dos dados (amostrados com reposição usando o método *bootstrap*) e combinando suas previsões por votação (classificação) ou média (regressão).

Random Forests. Random Forests [Breiman, 2001] são a realização mais bem-sucedida do bagging com árvores de decisão. A ideia adicional é introduzir aleatoriedade também na seleção de atributos: em cada nó, em vez de considerar todos os d atributos, considera-se apenas um subconjunto aleatório de $\sqrt{d}$ (para classificação) ou $d/3$ (para regressão) atributos candidatos.

Esse mecanismo de dupla aleatorização (amostras e atributos) produz árvores mais diversas e menos correlacionadas entre si, o que resulta em cancelamento mais eficaz dos erros individuais.



Random Forest: adiciona seleção aleatória de atributos em cada nó

Figura 11: Arquitetura do bagging / Random Forest. Múltiplas árvores são treinadas em amostras bootstrap independentes; suas previsões são combinadas por votação ou média, reduzindo a variância em relação a uma única árvore.

Boosting. O *boosting* adota uma estratégia diferente: em vez de treinar modelos independentes, treina-os *sequencialmente*, onde cada modelo foca nos exemplos que os anteriores erraram. O algoritmo mais clássico é o *AdaBoost*: exemplos classificados incorretamente recebem peso maior na rodada seguinte, forçando o próximo modelo a se concentrar neles.

O *Gradient Boosting* (e sua implementação eficiente, *XGBoost* [Chen and Guestrin, 2016]) generaliza essa ideia: cada novo modelo é treinado para corrigir os *resíduos* (erros) do ensemble acumulado até então. O resultado é uma sequência de modelos simples que, somados, produzem um modelo muito poderoso.

A Tabela 3 resume as principais diferenças entre os métodos de ensemble.

Tabela 3: Comparação entre métodos ensemble quanto ao componente do erro que atacam e à sua estratégia de combinação.

Método	Reduz principalmente	Estratégia
Bagging	Variância	Modelos paralelos em amostras bootstrap
Random Forest	Variância (mais que bagging)	Bagging + aleatoriedade em atributos
AdaBoost	Viés	Modelos sequenciais com reponderação
Gradient Boosting	Viés; pode controlar variância com regularização	Modelos sequenciais para corrigir resíduos

Boosting também pode sofrer overfitting

Embora o boosting seja frequentemente apresentado como uma técnica de redução de viés, modelos boosted também podem sofrer overfitting se forem muito profundos, numerosos ou pouco regularizados. Implementações modernas como XGBoost e LightGBM incorporam regularização e amostragem de dados por esse motivo. Sempre avalie o modelo boosted tanto no treino quanto na validação.

Micro-checkpoint

Árvores de decisão têm alta variância; modelos lineares simples têm alto viés. Como cada um dos métodos ensemble (bagging e boosting) ataca um desses dois problemas? Por que Random Forest funciona melhor que bagging de árvores simples?

Pergunta 5

Um conjunto de dados tem 1.000 exemplos e 20 atributos. Um único Decision Tree treinado até a folha (sem poda) obtém 100% de acurácia no treino e 74% na validação. Um Random Forest com 100 árvores obtém 88% de acurácia na validação. **(a)** Por que o Random Forest generaliza melhor? **(b)** Qual mecanismo do Random Forest é responsável por diversificar as árvores e reduzir sua correlação? **(c)** Se você quisesse melhorar ainda mais o desempenho, como compararia Random Forest com Gradient Boosting para este problema?

6 Redes Neurais Artificiais

6.1 Do perceptron à rede multicamadas

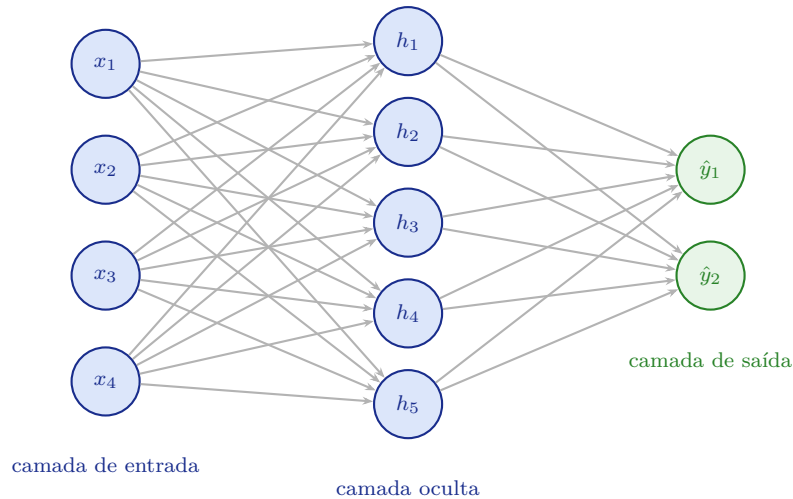
O *perceptron* é o bloco básico de construção das redes neurais. Ele recebe um vetor de entradas $\mathbf{x}$, computa uma combinação linear $z = \mathbf{w}^\top \mathbf{x} + b$, e aplica uma *função de ativação* σ para produzir uma saída:

$$\text{saída} = \sigma(\mathbf{w}^\top \mathbf{x} + b)$$

Com a função de ativação como degrau ($\sigma(z) = \mathbf{1}[z \geq 0]$), o perceptron é um classificador linear. Com a sigmoide como ativação, ele é equivalente à regressão logística. O perceptron simples é, portanto, um modelo linear, com a mesma limitação de não conseguir resolver problemas não linearmente separáveis.

O avanço crucial é o *perceptron multicamadas* (*multilayer perceptron*, MLP), que empilha múltiplas camadas de perceptrons. A Figura 12 ilustra uma rede com camada de entrada, uma camada oculta e uma camada de saída.

O poder de representação do MLP vem da não-linearidade: sem funções de ativação não lineares, qualquer composição de camadas seria equivalente a uma única camada linear. Com não-linearidade, camadas ocultas podem aprender *representações intermediárias* dos dados, uma forma automática de engenharia de características.



$$\mathbf{h} = \sigma(W^{(1)}\mathbf{x} + \mathbf{b}^{(1)}) \quad \hat{\mathbf{y}} = \text{softmax}(W^{(2)}\mathbf{h} + \mathbf{b}^{(2)})$$

Figura 12: Arquitetura de um perceptron multicamadas (MLP) com 4 entradas, 5 neurônios ocultos e 2 saídas. Cada conexão possui um peso aprendível. A camada oculta aplica uma transformação não linear σ , permitindo ao modelo aprender fronteiras de decisão não lineares.

Por que redes neurais são universais

O *teorema de aproximação universal* ([Hornik, 1991, Cybenko, 1989]) afirma que um MLP com uma única camada oculta suficientemente larga pode aproximar qualquer função contínua com precisão arbitrária. Isso significa que a classe de hipóteses dos MLPs é extraordinariamente expressiva. A questão prática, no entanto, não é *se* um MLP pode representar a função, mas se ele consegue aprendê-la a partir de dados finitos e com gradiente descendente.

6.2 Funções de ativação

A escolha da função de ativação influencia fortemente a capacidade e a eficiência do treinamento. As mais usadas são:

- Sigmoid: $\sigma(z) = \frac{1}{1+e^{-z}} \in (0, 1)$. Historicamente importante, mas sofre de *vanishing gradient* em redes profundas.
- Tanh: $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \in (-1, 1)$. Similar à sigmoide, mas centrada em zero; geralmente preferível.
- ReLU (*Rectified Linear Unit*): $\text{ReLU}(z) = \max(0, z)$. Simples, eficiente, e o padrão atual na maioria das arquiteturas. Não sofre de *vanishing gradient* para $z > 0$.
- Softmax (camada de saída para classificação multiclasse): $\text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$. Normaliza as saídas para uma distribuição de probabilidade sobre K classes.

O problema do gradiente que desaparece

A sigmoide comprime qualquer entrada para $(0, 1)$, e sua derivada máxima é 0,25. Em uma rede profunda, o gradiente é multiplicado por essa derivada em cada camada durante a retropropagação. Com 10 camadas, o gradiente é multiplicado por no máximo $0,25^{10} \approx 10^{-6}$: torna-se praticamente zero. Isso faz com que os pesos das primeiras camadas aprendam muito lentamente ou nada. Esse problema, chamado de *vanishing gradient*, foi a principal barreira ao treinamento de redes profundas antes da adoção da ReLU.

6.3 Retropropagação

Como treinar os parâmetros de uma rede neural? O mesmo princípio do gradiente descendente se aplica, mas agora é necessário calcular o gradiente do custo em relação a *todos* os pesos da rede. O algoritmo que faz isso de forma eficiente é a *retropropagação* (*backpropagation*).

A retropropagação não é um algoritmo de otimização; é um algoritmo de cálculo de gradientes. Ele aplica a *regra da cadeia do cálculo* de forma sistemática, propagando o erro da camada de saída de volta para a camada de entrada:

$$\frac{\partial J}{\partial w_{jk}^{(l)}} = \frac{\partial J}{\partial a_j^{(l)}} \cdot \frac{\partial a_j^{(l)}}{\partial w_{jk}^{(l)}}$$

onde $a_j^{(l)}$ é a ativação do neurônio j na camada l e $w_{jk}^{(l)}$ é o peso que conecta o neurônio k da camada $l - 1$ ao neurônio j da camada l .

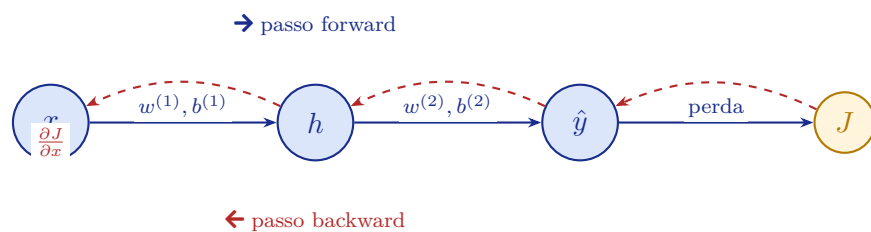


Figura 13: Intuição da retropropagação. No passo *forward*, os dados são propagados da esquerda para a direita para computar a perda J . No passo *backward*, os gradientes são propagados da direita para a esquerda via regra da cadeia, calculando $\frac{\partial J}{\partial w}$ para cada peso.

A retropropagação calcula os gradientes de forma exata e eficiente em $O(W)$, onde W é o número total de pesos da rede. Uma vez calculados, os gradientes são usados no gradiente descendente estocástico (SGD) para atualizar os pesos:

$$w_{jk}^{(l)} \leftarrow w_{jk}^{(l)} - \alpha \cdot \frac{\partial J}{\partial w_{jk}^{(l)}}$$

Retropropagação não é magia

Um equívoco comum é imaginar que a retropropagação “ajusta os pesos inteligentemente”. Na realidade, ela faz uma coisa simples: aplicar a regra da cadeia sistematicamente para calcular derivadas parciais. O que há de elegante é a eficiência: em vez de estimar gradientes por diferenças finitas (custoso), a retropropagação os calcula de forma exata em tempo linear no número de pesos.

A atualização dos pesos em si é o mesmo gradiente descendente da regressão linear. A única novidade é o cálculo eficiente dos gradientes numa função composta por muitas camadas.

6.4 Aprendizado profundo: uma visão geral

O *aprendizado profundo* (*deep learning*) é o nome dado ao uso de redes neurais com muitas camadas ocultas. A profundidade permite que as camadas intermediárias aprendam representações hierárquicas dos dados: na visão computacional, por exemplo, as primeiras camadas detectam bordas, as intermediárias detectam formas, e as últimas detectam objetos.

O sucesso do aprendizado profundo a partir de 2012 (com o AlexNet em ImageNet) foi impulsionado por três fatores:

- Dados: disponibilidade de conjuntos de dados maciços (ImageNet, Common Crawl, etc.).
- Computação: GPUs que permitem treinamento de redes grandes em tempo razoável.
- Algoritmos: melhores funções de ativação (ReLU), regularização (Dropout), normalização de lotes (*batch normalization*), e arquiteturas especializadas.

As arquiteturas mais influentes incluem:

- Redes convolucionais (CNNs): especializadas em dados com estrutura espacial (imagens).
- Redes recorrentes (RNNs / LSTMs): especializadas em sequências (texto, áudio, séries temporais).
- Transformers: arquitetura baseada em atenção, dominante em processamento de linguagem natural; base dos modelos GPT, BERT e Claude.

Aprendizado profundo e engenharia de características

Uma das contribuições mais importantes do aprendizado profundo é automatizar a engenharia de características: em vez de extrair manualmente $f_i(x)$, a rede aprende automaticamente quais características são relevantes como parte do treinamento. Isso explica por que modelos profundos, com dados suficientes, frequentemente superam modelos “rasos” com características manuais cuidadosamente projetadas.

Redes profundas não são solução universal

Redes neurais profundas têm alta capacidade, mas essa capacidade só é útil quando há dados, computação e regularização suficientes. Em bases pequenas ou tabulares, modelos mais simples — como regressão logística, Random Forests ou Gradient Boosting — frequentemente apresentam desempenho equivalente ou superior, com muito menos custo computacional e maior interpretabilidade. A escolha do modelo deve ser guiada pelo problema, não pela complexidade.

Micro-checkpoint

Explique em uma frase o que a retropropagação faz. Descreva o papel da função de ativação numa rede neural: o que aconteceria se todas as ativações fossem lineares? Qual arquitetura de rede neural seria mais adequada para classificar imagens de raio-X médico?

Pergunta 6

Considere uma rede neural com uma camada oculta de 10 neurônios ReLU treinada para classificação binária (spam/não-spam). O modelo obtém 99% de acurácia no treino mas apenas 65% na validação. **(a)** Qual problema está ocorrendo? **(b)** Cite três técnicas específicas de treinamento de redes neurais que poderiam melhorar a generalização. **(c)** Como o conceito de regularização (Seção 4) se aplica a redes neurais?

7 Aprendizado Não Supervisionado

7.1 Quando não temos rótulos

Nos problemas de aprendizado supervisionado, cada exemplo de treinamento vem acompanhado de um rótulo y . Na prática, rotular dados é caro, demorado e muitas vezes impraticável em larga escala. O *aprendizado não supervisionado* aborda o problema de extrair estrutura de dados *sem rótulos*.

As tarefas mais comuns são:

- Agrupamento (*clustering*): encontrar grupos naturais nos dados.
- Redução de dimensionalidade: representar os dados em um espaço de menor dimensão, preservando estrutura relevante.
- Estimação de densidade: modelar a distribuição $p(\mathbf{x})$.

7.2 k -means

O algoritmo k -means é o método de agrupamento mais simples e amplamente usado. Dado um conjunto de pontos $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(n)}\}$ e um número de grupos k , o k -means encontra k *centróides* $\{\boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_k\}$ que minimizam a soma das distâncias quadráticas de cada ponto ao seu centróide mais próximo:

$$J(\{C_i\}, \{\boldsymbol{\mu}_i\}) = \sum_{i=1}^k \sum_{\mathbf{x} \in C_i} \|\mathbf{x} - \boldsymbol{\mu}_i\|^2$$

O algoritmo alterna entre dois passos até convergência:

Algoritmo 1 *k*-means

- 1: Inicializar k centróides $\mu_1, \dots, \mu_k$ (e.g., k pontos aleatórios do conjunto)
- 2: **repeat**
- 3: **Passo E (atribuição)**: atribuir cada ponto $\mathbf{x}^{(i)}$ ao grupo cujo centróide é mais próximo:

$$c^{(i)} \leftarrow \arg \min_j \|\mathbf{x}^{(i)} - \mu_j\|^2$$

- 4: **Passo M (atualização)**: recomputar cada centróide como a média dos pontos atribuídos a ele:

$$\mu_j \leftarrow \frac{1}{|C_j|} \sum_{\mathbf{x} \in C_j} \mathbf{x}$$

- 5: **until** as atribuições $c^{(i)}$ não mudarem
 - 6: **return** centróides $\{\mu_j\}$ e atribuições $\{c^{(i)}\}$
-

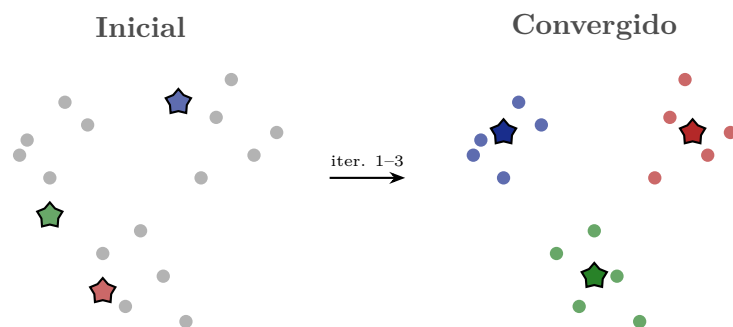


Figura 14: O algoritmo *k*-means com $k = 3$. Partindo de centróides iniciais aleatórios (estrelas), o algoritmo alterna entre atribuição de pontos ao centróide mais próximo e recálculo dos centróides, convergindo para um agrupamento estável.

7.2.1 Exemplo resolvido

Aplique *k*-means com $k = 2$ ao conjunto $\{(1, 1), (1, 2), (5, 4), (5, 5)\}$. Centróides iniciais: $\mu_1 = (1, 1)$ e $\mu_2 = (5, 4)$.

Iteração 1 – Passo E:

- $(1, 1)$: dist. a $\mu_1 = 0$; dist. a $\mu_2 = \sqrt{32} \approx 5,7$. Atribui ao grupo 1.
- $(1, 2)$: dist. a $\mu_1 = 1$; dist. a $\mu_2 = \sqrt{25} = 5$. Atribui ao grupo 1.
- $(5, 4)$: dist. a $\mu_1 = \sqrt{25} = 5$; dist. a $\mu_2 = 0$. Atribui ao grupo 2.
- $(5, 5)$: dist. a $\mu_1 = \sqrt{32} \approx 5,7$; dist. a $\mu_2 = 1$. Atribui ao grupo 2.

Iteração 1 – Passo M:

$$\mu_1 = \frac{(1, 1) + (1, 2)}{2} = (1, 1,5) \quad \mu_2 = \frac{(5, 4) + (5, 5)}{2} = (5, 4,5)$$

Iteração 2 – Passo E: as atribuições não mudam. Algoritmo converge.

Interpretação: os dois grupos naturais $\{(1, 1), (1, 2)\}$ e $\{(5, 4), (5, 5)\}$ foram corretamente identificados em duas iterações.

Limitações do k -means

O k -means tem limitações importantes: **(a)** requer que k seja fornecido a priori; **(b)** converge para um mínimo local que depende da inicialização (o algoritmo *k-means++* melhora a inicialização); **(c)** assume grupos de forma esférica e variância similar; falha com grupos alongados, em forma de lua crescente, ou com densidade muito diferente.

7.3 Análise de Componentes Principais (PCA)

A *análise de componentes principais* (PCA, *Principal Component Analysis*) é o método de redução de dimensionalidade mais clássico. Dado um conjunto de dados em $\mathbb{R}^d$, o PCA encontra uma projeção linear para um espaço de dimensão $k < d$ que *maximiza a variância preservada* (equivalentemente, minimiza o erro de reconstrução).

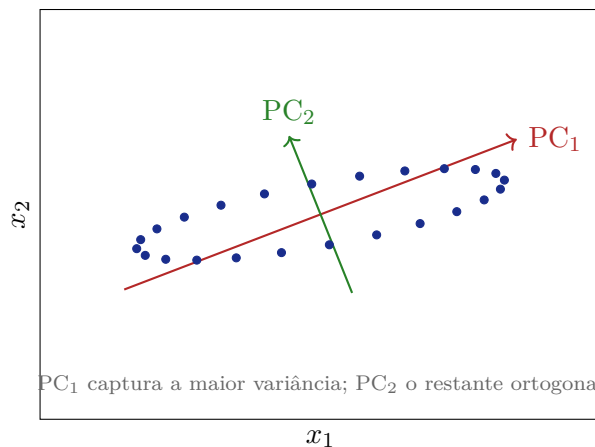


Figura 15: PCA em 2D. Os dados formam uma nuvem elíptica. A primeira componente principal (PC_1) aponta na direção de maior variância; a segunda (PC_2) é perpendicular a ela. Projetar os dados sobre apenas PC_1 reduz a dimensão de 2 para 1, com mínima perda de informação.

Intuitivamente, o PCA encontra as direções do espaço de características ao longo das quais os dados variam mais. Projetar os dados sobre as k primeiras componentes principais retém a maior parte da variância com apenas k dimensões.

O PCA tem múltiplas aplicações práticas:

- Visualização: projetar dados de alta dimensão para 2D ou 3D.
- Pré-processamento: reduzir dimensionalidade antes de aplicar outro algoritmo de AM, acelerando o treinamento.
- Remoção de ruído: as componentes de menor variância frequentemente capturam ruído; descartá-las pode melhorar o sinal.
- Compressão: representar imagens ou documentos de forma compacta.

PCA e redes neurais

O PCA computa uma projeção linear ótima. Autocodificadores (*autoencoders*) são redes neurais treinadas para comprimir e reconstruir dados. Com ativações lineares, um autocodificador de uma camada é equivalente ao PCA. Com ativações não lineares, o autocodificador aprende projeções não lineares, uma generalização poderosa do PCA.

Micro-checkpoint

Qual a diferença fundamental entre aprendizado supervisionado e não supervisionado? O que o k -means minimiza? Por que é necessário especificar k a priori, e como você poderia escolher um bom valor de k sem conhecimento prévio?

Pergunta 7

Uma empresa de varejo quer segmentar seus clientes em grupos para personalizar campanhas de marketing. Os dados disponíveis incluem: valor médio de compra, frequência de visitas mensais, categorias de produtos comprados (como um vetor de 50 dimensões representando proporção de compras em cada categoria) e informações demográficas. **(a)** Argumente por que esta é uma tarefa de aprendizado não supervisionado, e não de aprendizado supervisionado. **(b)** Quais são as limitações do k -means para este problema? **(c)** Por que seria útil aplicar PCA antes do k -means neste caso?

8 Resumo

A Tabela 4 sintetiza os algoritmos de AM estudados nesta leitura, destacando paradigma, tipo de função aprendida, perfil viés-variância e observações práticas.

Visão geral do aprendizado de máquina

O AM é o estudo de algoritmos que aprendem a partir de dados. O framework supervisionado formaliza o aprendizado como minimização de um risco empírico num espaço de hipóteses, sujeito à restrição implícita de generalização. O dilema viés-variância capta a tensão fundamental entre modelos simples (que generalizam mas erram sistematicamente) e modelos complexos (que ajustam bem o treino mas podem não generalizar). A regularização, a validação cruzada e a divisão adequada dos dados são os instrumentos práticos para navegar esse dilema.

Modelos lineares são simples, eficientes e interpretáveis, mas limitados. Árvores de decisão são interpretáveis mas instáveis; métodos ensemble as tornam robustas. Redes neurais profundas são universalmente expressivas e aprendem características automaticamente, ao custo de requisitos maiores de dados, computação e conhecimento de treinamento. Cada família de modelos tem seu lugar, e a escolha deve ser guiada pela natureza do problema, pelo volume e qualidade dos dados, e pelos requisitos de interpretabilidade e desempenho.

Tabela 4: Síntese dos algoritmos estudados nas notas.

Algoritmo	Paradigma	O que aprende	Viés/Variância	Escalabilidade / Notas
Regressão linear	Supervisionado	Função linear de predição	Alto viés, baixa variância	Solução analítica para d pequeno; SGD para d grande
Regressão logística	Supervisionado	Fronteira linear (classificação)	Alto viés, baixa variância	Eficiente; interpretável; base do softmax
Árvore de decisão	Supervisionado	Particionamento hierárquico	Baixo viés, alta variância	Interpretável; instável; tende a overfitting sem poda
Random Forest	Supervisionado	Ensemble de árvores	Reduz variância	Robusto; pouco sensível a hiperparâmetros; escalável
Gradient Boosting	Supervisionado	Ensemble sequencial	Reduz viés e variância	Muito poderoso; mais sensível a hiperparâmetros
MLP / Deep Learning	Supervisionado	Representação hierárquica	Depende da arquitetura	Alta capacidade; requer muitos dados; treino custoso
k -means	Não supervisionado	Centróides de grupos	–	Sensível a inicialização e a k ; assume clusters esféricos
PCA	Não supervisionado	Projeção linear de maior variância	–	Projeção linear ótima; eficiente; base de muitas técnicas

Pergunta 8

As notas de AR compararam Q-learning tabular (exato, mas não escala) com Q-learning aproximado (generaliza, mas sem garantia de Q^*). Trace um paralelo preciso com a comparação entre árvores de decisão (exatas sobre o treino, sem regularização) e Random Forests (generalizadas, mas sem garantia de ótimo global). O que esses dois casos têm em comum conceitualmente? O que os distingue?

9 Atividades Intra-Classe

As atividades abaixo devem ser resolvidas em grupo, com foco em explicar o raciocínio passo a passo, e não apenas em obter a resposta final.

9.1 Aquecimento

Antes das atividades principais, responda rapidamente:

- Qual a diferença entre um parâmetro (como os pesos $\mathbf{w}$) e um hiperparâmetro (como λ na regularização)? O que é ajustado durante o treinamento e o que é ajustado durante a validação?
- Um modelo obteve erro de treino = 0,02 e erro de validação = 0,85. Qual problema você diagnosticaria? E se o erro de treino fosse 0,80 e o de validação 0,82?
- Cite a diferença entre MSE e entropia cruzada. Para qual tarefa cada uma é mais adequada?
- Explique a diferença entre bagging e boosting em uma frase cada.
- Por que inicializar todos os pesos de uma rede neural com zero é problemático? (*Dica:* pense no que acontece com o gradiente se todos os neurônios de uma camada tiverem os mesmos pesos.)
- Em k -means, o que o algoritmo está minimizando? O que garante convergência?

Explique como se fosse um professor

Explique para um colega a diferença entre viés e variância usando apenas:

- a analogia de um arqueiro num campo de tiro;
- o comportamento do modelo com dados de treino vs. dados de validação;
- um exemplo concreto com regressão linear de grau 1 vs. grau 15;
- por que um modelo com alto viés e um com alta variância cometem erros muito diferentes.

Evite fórmulas. Se você não consegue explicar sem equações, revise a Seção 4.

9.2 Regressão linear e gradiente descendente

Considere um problema de regressão com os dados a seguir:

Exemplo	Área (x_1 , m ²)	Quartos (x_2)	Preço (y , R\$ mil)
1	50	1	200
2	80	2	300
3	120	3	450
4	60	2	250

- Escreva a hipótese do modelo: $h(\mathbf{x}) = w_0 + w_1x_1 + w_2x_2$. Calcule o MSE para os pesos iniciais $w_0 = 0$, $w_1 = 3$, $w_2 = 10$.
- Calcule o gradiente $\frac{\partial J}{\partial w_1}$ e $\frac{\partial J}{\partial w_2}$ com os pesos acima. (Use $\frac{\partial J}{\partial w_j} = \frac{2}{n} \sum_i (\hat{y}_i - y_i)x_{ij}$.)
- Execute um passo de gradiente descendente com $\alpha = 0,0001$ e reporte os pesos atualizados.
- Após o passo, o MSE aumentou ou diminuiu? Por que isso era esperado?

9.3 Entropia e construção de árvores

Considere um conjunto de 12 exemplos de e-mails: 8 spam e 4 legítimos. Dois atributos candidatos para a divisão raiz:

- **Atributo A** (“oferta” presente): $S_{\text{sim}} = \{6\text{spam}, 1\text{leg}\}$, $S_{\text{não}} = \{2\text{spam}, 3\text{leg}\}$.
- **Atributo B** (enviado de noite): $S_{\text{sim}} = \{4\text{spam}, 2\text{leg}\}$, $S_{\text{não}} = \{4\text{spam}, 2\text{leg}\}$.

- Calcule a entropia do conjunto raiz $H(S)$.
- Calcule o ganho de informação $IG(S, A)$ e $IG(S, B)$.
- Qual atributo o algoritmo de árvore escolheria? Justifique intuitivamente.
- O Atributo B tem IG igual a zero. O que isso diz sobre o poder discriminativo desse atributo? (Calcule para confirmar.)

9.4 Diagnóstico de generalização

Para cada cenário abaixo, diagnostique o problema (underfitting, overfitting, ou adequado), identifique o provável desequilíbrio viés-variância e proponha uma ação corretiva específica.

- Um modelo de regressão linear simples ($d = 1$, sem termos polinomiais) para prever o consumo de energia em função da temperatura, com erro de treino = 320 e erro de validação = 335.
- Uma Random Forest com 500 árvores, profundidade máxima ilimitada, treinada em 200 exemplos, com erro de treino = 0 e erro de validação = 0,45 (acurácia de 55%).
- Uma regressão logística com regularização ℓ_2 , $\lambda = 1000$, com acurácia de treino = 53% e de validação = 52%.
- Um MLP com 5 camadas ocultas de 1.000 neurônios, treinado por 500 épocas, com perda de treino = 0,001 e perda de validação = 2,8.

9.5 k -means: execução manual

Aplique o algoritmo k -means com $k = 3$ ao conjunto de pontos em 2D (coordenadas inteiras):

Ponto	Coordenadas
A	(1, 1)
B	(1, 2)
C	(2, 1)
D	(8, 8)
E	(9, 8)
F	(8, 9)
G	(4, 5)
H	(5, 4)
I	(5, 5)

Centróides iniciais: $\mu_1 = (1, 1)$, $\mu_2 = (4, 5)$, $\mu_3 = (8, 8)$.

- (a) Execute a iteração 1 completa: passo E (atribuição) e passo M (atualização dos centróides). Use distância euclidiana.
- (b) Execute a iteração 2 e verifique se o algoritmo convergiu.
- (c) O custo J diminuiu da iteração 1 para a iteração 2? Por que o custo nunca aumenta entre iterações no k -means?

9.6 Perguntas para discussão em sala

Pergunta 9

Considere o seguinte cenário: você tem 100.000 e-mails rotulados (spam/legítimo) e precisa escolher entre três modelos: regressão logística, Random Forest e MLP com 3 camadas. Descreva o processo completo de desenvolvimento e avaliação, desde a divisão dos dados até a implantação do modelo. Que critérios usaria para a escolha final entre os três modelos?

Pergunta 10

O dilema viés-variância implica que não existe um modelo universalmente melhor. No entanto, nas competições Kaggle de AM, Gradient Boosting (XGBoost/-LightGBM) e redes neurais dominam consistentemente. Como reconciliar o teorema “no free lunch” com o sucesso prático desses métodos específicos em tantos problemas? (*Dica:* considere que competições Kaggle não amostram uniformemente todos os problemas possíveis de AM; elas concentram certos tipos de dados, métricas e tarefas.)

Pergunta 11

Um sistema de concessão de crédito usa um modelo de AM para decidir se um empréstimo deve ser aprovado. O modelo foi treinado com dados históricos de uma instituição financeira. **(a)** Quais riscos surgem se a hipótese de aprendizagem não for satisfeita nesse contexto? **(b)** Por que a acurácia pode ser uma métrica inadequada? **(c)** Que considerações éticas surgem quando os dados históricos refletem vieses sociais existentes?

Pergunta 12

As notas de AR descrevem o Q-learning com aproximação linear como um gradiente descendente sobre o erro temporal. À luz do que foi estudado sobre AM, identifique: **(a)** o “conjunto de treinamento” equivalente no Q-learning; **(b)** o “rótulo” (y) equivalente; **(c)** de que forma o problema de generalização aparece no Q-learning com aproximação, e por que ele assume uma forma diferente do overfitting supervisionado clássico; **(d)** que tipo de regularização seria natural aplicar ao Q-learning com aproximação.

Referências

- Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. doi: 10.1023/A:1010933404324.
- Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, San Francisco, CA, USA, 2016. ACM. doi: 10.1145/2939672.2939785.
- George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems*, 2(4):303–314, 1989. doi: 10.1007/BF02551274.
- Kurt Hornik. Approximation capabilities of multilayer feedforward networks. *Neural Networks*, 4(2):251–257, 1991. doi: 10.1016/0893-6080(91)90009-T.